Beyond Task Completion: Auditing Evolving Tool Libraries in Agentic AI

Anonymous Author(s)

Abstract

Agentic AI systems increasingly create and reuse executable tools across tasks. Once such tools persist, the object being evaluated is no longer only a single model response; it is also a growing software artifact that can be reused, duplicated, composed, or broken by later actions. Standard task-completion benchmarks observe only the immediate answer and therefore miss a central trustworthiness question: is the tool library left behind reliable enough to support future work?

We introduce EvolveTool-Bench, a diagnostic framework for evaluating evolving tool libraries as auditable software artifacts. The benchmark organizes tasks into seed, gap, variant, composition, regression, and adversarial phases, and records verified task completion, tool creation, reuse, correct reuse, incorrect reuse, and per-run audit traces. The framework is deliberately not a leaderboard: its purpose is to help researchers see which part of a tool-evolving agent succeeds or fails.

In the current strict pilot, we evaluate five representative protocols over the deterministically verified subset of the benchmark: 8 sessions, 51 verified task decisions per system pass, 3 seeds, and 5 protocols. The strict run does not establish task-completion superiority for any tool-creation protocol; the pre-specified improvement contrasts have confidence intervals that do not support positive TC gains. Rather than treating this as a leaderboard outcome, we use it to motivate the benchmark's central design choice: trustworthy evaluation should report verifier coverage, exclusion accounting, reuse behavior, and artifact summaries alongside task completion. We release the benchmark design, audit schema, and strict-subset summary manifests to support reproducible inspection and future extensions.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → **Software verification and validation**.

Keywords

agentic AI, tool use, evaluation, benchmark, auditability, verification

1 Introduction

A growing class of agents does more than call a fixed tool set. These systems write Python functions, API clients, parsers, analysis scripts, or richer skill bundles, and then carry those artifacts forward into later tasks. VOYAGER learns reusable code skills in Minecraft [Wang et al.(2023)]; CREATOR separates tool creation from tool use [Qian et al.(2023)]; LLMs as Tool Makers and ToolMaker study agents that construct tools for other agents to call [Cai et al.(2023), Wölflein et al.(2025)]; EvoSkill and CoEvoSkills treat skills as evolving units of agent knowledge [Alzubi et al.(2026), Zhang et al.(2026)]. In all of these settings, the agent leaves behind a software artifact.

That artifact matters. A generated library can solve the next task while still containing brittle parsing logic, redundant functions, untested assumptions, or changed behavior that breaks earlier capabilities. A task-completion score tells us whether an answer was accepted. It does not tell us whether the agent created a reusable tool, reused the right tool, duplicated an existing one, or propagated a bug through reuse. For deployed or long-running agents, those differences are not cosmetic; they are the difference between a one-off success and an auditable system.

EvolveTool-Bench is built around this distinction. It evaluates agents that may create tools during a session and asks not only "did the task pass?" but also "what happened to the library?" Each session has known dependency structure: seed tasks test provided tools, gap tasks invite new capabilities, variant tasks test reuse, composition tasks test chaining, regression tasks revisit prior behavior, and adversarial tasks probe edge cases. The resulting traces let a researcher inspect whether a failure came from synthesis, reuse, verification, composition, or regression.

The benchmark design contains 99 tasks across three domains. Of these, the data-transformation and numerical domains currently expose deterministic task verifiers; the API-orchestration session is part of the benchmark design but excluded from the main TC denominator until its verifiers are complete. Unverified tasks are reported separately and not credited as successes. This narrows the empirical claim of any single submission while tightening the evaluation claim of the framework as a whole.

Our contributions are:

(1) **A diagnostic protocol for evolving tool libraries.** Sessions are structured so that creation, reuse, composition, regression, and adversarial behavior can be inspected separately.
(2) **Artifact-level measurements beyond TC.** We define and log tool creation, raw reuse, correct reuse, incorrect reuse, utilization, composition, and regression signals alongside verified task completion.
(3) **Verifier-aware reporting.** The strict analysis separates the benchmark's full design scope from the currently verified subset, preventing unverified tasks from inflating task-completion claims.
(4) **An audit-trace view of agent evaluation.** Runs preserve task outcomes, tools created, tools used, reuse outcomes, and verifier status so that aggregate scores can be traced back to concrete artifacts.

The main result is not a protocol ranking. On the current strict subset, the task-completion comparison is inconclusive. The result is that a benchmark for tool-evolving agents must make the state of the tool library visible even when immediate pass rates differ by only a few points.

2 Related Work

Tool and skill creation. Systems such as CREATOR, LLMs as Tool Makers, ToolMaker, EvoSkill, and CoEvoSkills all study agents that create tools or skills rather than merely selecting from a fixed tool set [Qian et al.(2023), Cai et al.(2023), Wölflein et al.(2025), Alzubi et al.(2026), Zhang et al.(2026)]. Their evaluations typically focus on downstream success, per-tool validation, or skill-level pass rates. EvolveTool-Bench is complementary: it asks what happens to the library over time, especially under reuse, composition, and regression pressure.

Agent and tool-use benchmarks. ToolBench and AgentBench evaluate tool-using agents with predefined environments or APIs [Qin et al.(2023), Liu et al.(2024)]. Vending-Bench stresses long-horizon agent behavior with a fixed operational environment [Backlund and Petersson(2025)]. These benchmarks are valuable, but the tool inventory is generally not an artifact produced by the agent. Our setting differs because the generated library is part of the system under evaluation.

Skill benchmarks and library learning. SkillsBench shows that skills can help some tasks and hurt others, especially when the skills are self-generated [Li et al.(2026)]. Program-synthesis work such as DreamCoder studies learned libraries and evaluates their effect on solve rates or compression [Ellis et al.(2021)]. The question we address sits between these traditions: when an LLM agent accumulates executable artifacts, can we tell whether the resulting library is correct, reusable, and stable?

Quality of generated code. Tool-Genesis, NoFunEval, and related code-evaluation work measure compliance, unit-test performance, maintainability, or non-functional requirements for generated code [Xia et al.(2026), Singhal et al.(2024)]. EvolveTool-Bench borrows the software-engineering lens but applies it at session and library granularity. A single generated function can look reasonable in isolation while still making the library harder to use.

3 Benchmark Design

3.1 Sessions and task roles

A session is a fixed sequence of 11 tasks with shared library state. The sequence is intentionally simple: it creates controlled opportunities for a system to use, create, reuse, compose, and preserve tools. Table 1 summarizes the task roles.

The benchmark contains three domains and nine sessions: five data-transformation sessions, one API-orchestration session, and three numerical-computation sessions. Data-transformation sessions use proprietary formats such as binary encodings and schema-like validators. Numerical sessions use curve fitting, signal processing, and constrained optimization tasks. The API session uses a mock service with HMAC authentication and cursor handling. The full benchmark contains 99 tasks, but the strict analysis in this paper uses only tasks with deterministic verifiers.

3.2 Verification policy

Verification is part of the contribution, not a footnote. A task contributes to strict TC only if it has an explicit expected output or a custom verifier. If a task lacks deterministic verification, it is reported as unverified and excluded from the strict TC denominator. It is not counted as passed because the agent produced a long or plausible answer. This policy avoids the common failure mode in agent benchmarks where format-looking output is mistaken for correctness.

Table 1: Task roles in each session. The fixed order lets the harness attribute behavior to creation, reuse, composition, regression, or edge-case handling rather than reporting one undifferentiated pass rate.

Role	Count	Diagnostic purpose
Seed	3	Use provided tools; establish baseline behavior.
Gap	2	Solve a task requiring a missing capability.
Variant	2	Recognize that a prior capability can be reused.
Compose	1	Chain multiple capabilities or tools.
Regress	1	Revisit an earlier contract after the library has changed.
Adversarial	2	Handle edge cases or malformed inputs.

Table 2: Verifier coverage of the strict subset by task role (eight verified-subset sessions; API-orchestration excluded). Seed tasks deliberately carry no graded outcome and are not part of the TC denominator. The roles that the diagnostic framework leans on – gap, variant, regress – are near-fully covered; compose and adversarial have partial coverage.

Role	Verified	Total	Coverage
Seed (warm-up, ungraded)	0	24	—
Gap	15	16	93.8%
Variant	15	16	93.8%
Compose	4	8	50.0%
Regress	8	8	100.0%
Adversarial	9	16	56.2%
Strict-subset total (graded)	51	64	79.7%

The current strict pilot covers 8 sessions and 51 verified task decisions per complete system pass. The API-orchestration session is excluded from the main strict TC analysis because its task verifiers are not yet complete. We keep the API session in the benchmark design because it is an important tool-use domain, but we do not use it to support the main numerical claims in this submission.

Table 2 shows how that 51-task verified subset is distributed across task roles. Two facts matter here. First, seed tasks carry no graded outcome by design: they exist to expose the agent to the provided library before any creation, so they have no expected output or verifier. Second, the diagnostic-critical roles – gap, variant, regress – are essentially fully covered (94–100%); compose and adversarial are partially covered. The strict subset therefore exercises the roles the framework relies on for its substantive claims, while remaining honest about where verifier work is still needed.

3.3 Metrics and audit trace

Verified task completion (TC).. TC is the fraction of verified tasks whose outputs satisfy the task verifier. It measures user-visible success on the verified subset and remains a necessary metric. We do not treat it as sufficient.

Tool creation and reuse. Before each task, the harness records the current library. After the task, it records newly created tools and tools used. Reuse is counted when a task invokes a tool that existed before the task began. We decompose reuse into:

- **correct reuse**: a prior tool was used and the task passed;
- **incorrect reuse**: a prior tool was used and the task failed.

This distinction is important because raw reuse can mean either useful abstraction or repeated reliance on a defective artifact.

Library-health diagnostics. The broader framework also supports redundancy, utilization, composition, regression stability, and capability-aligned hidden conformance tests. These diagnostics are part of the benchmark design and audit schema. In this strict-subset submission, we report only the signals that are aligned with the current verified manifests: TC, tools created, correct reuse, incorrect reuse, and reuse precision. We deliberately do not headline a full library-health or tool-quality ranking until the remaining verifier and conformance-test mappings are complete.

Audit trace. Each run is intended to emit a trace containing the task id, task role, verifier status, model, seed, pass/fail outcome, tools available before the task, tools used, tools created, artifact summaries, and LLM-call counts. The trace makes the benchmark inspectable: a reader can move from a table entry to the task and artifact that produced it.

4 Experimental Setup

The strict pilot evaluates five representative protocols. They are not meant to exhaust the design space; they provide enough variation to test whether the harness can distinguish tool-creation and reuse behavior.

(1) **No-Evolution.** The system receives the seed tools and never modifies the library. It is the lower-bound protocol for library evolution.
(2) **One-Shot.** When the system attempts tool creation, it adds a generated function without iterative repair or external validation. This tests unvalidated creation.
(3) **EvoSkill-style.** A strategy-level adaptation inspired by EvoSkill. It maintains natural-language skill hints rather than executable tools.
(4) **ToolMaker-style.** A protocol adaptation inspired by ToolMaker's diagnose-and-rewrite loop. It is not a native port; the original setting assumes reference tests and repository inputs that our harness does not provide.
(5) **CREATOR-style.** A decision/execution split inspired by CREATOR, where the system explicitly decides whether a tool should be created before executing the task.

All strict-subset runs use Claude Haiku 4.5 (snapshot `claude-haiku-4-5-20251001`) accessed via the Anthropic API with temperature 1.0 and the model default top-p. Each protocol is executed with three integer seeds (`0, 1, 2`) over eight verified-subset sessions, producing 24 session rows per protocol. Synthesised tools execute in an in-process sandbox with a five-second wall-clock limit; harness state is reset between sessions so library accumulation is per-session, not cross-session. The same Anthropic API key, decoding parameters, task stream, and verifier set are used for all protocols; only the tool-creation control flow varies. This is a pilot-scale analysis: it is adequate for exposing whether the current evidence supports a task-completion improvement claim; it is not adequate for declaring stable protocol rankings across models or domains.

Table 3: Strict verified-subset pilot. TC is computed only on tasks with deterministic verifiers; unverified tasks are reported separately and excluded from the TC denominator. Means and standard errors are over 24 session rows (3 seeds × 8 verified-subset sessions) using Claude Haiku 4.5.

System	TC	SE	Tools	Reuse precision
One-Shot	0.358	0.077	114	0.333
No-Evolution	0.337	0.064	0	0.333
EvoSkill-style	0.332	0.062	0	0.250
ToolMaker-style	0.292	0.054	18	0.375
CREATOR-style	0.287	0.048	111	0.333

Statistical procedure. For each pre-specified contrast we form per-session paired differences across the 24 strict-subset session rows (3 seeds × 8 sessions), bootstrap-resample those pairs B=10,000 times with seed 20260604, and report the bootstrap point estimate and 95% percentile CI. The three pre-specified contrasts in Table 4 are Benjamini–Hochberg adjusted. A contrast is marked *supported* only if its CI lies strictly above zero in the pre-specified positive direction.

Artifacts. The benchmark sources, harness, verifier definitions, canonical manifests (`run_manifest.jsonl`, `tool_manifest.jsonl`, `aggregate.json`, `audit_report.json`, `statistical_report.json`, `claims.json`), and the scripts that regenerate every table in this paper from disk are included in the anonymised supplement and will be released under an open licence on acceptance.

5 Results

5.1 Strict TC does not support a protocol winner

Across the five protocols, strict TC ranges from 0.287 (CREATOR-style) to 0.358 (One-Shot), with No-Evolution (0.337) and EvoSkill-style (0.332) in between (Table 3). The spread is small relative to the per-session standard errors and the heterogeneity across sessions.

The pre-specified improvement tests also do not support a positive task-completion claim (Table 4). One-Shot is not distinguishable from No-Evolution in the expected positive direction. ToolMaker-style does not improve over One-Shot. The Creator − One-Shot contrast is the only pre-specified comparison whose 95% CI excludes zero: −7.1 pp with CI [−13.9, −0.1]. Under the pre-specified positive test this does not support a TC-improvement claim for

Table 4: Pre-specified TC contrasts in the strict verified-subset pilot. Positive effects were expected; none support a TC-improvement claim. Values are absolute TC differences.

Contrast	Δ	95% CI	Improves?
OneShot − NoEvol	0.0208	[−0.0347, 0.0729]	No
Creator − OneShot	-0.0710	[−0.1389, −0.0012]	No
ToolMaker − OneShot	-0.0655	[−0.1443, 0.0174]	No

CREATOR-style; in the opposite direction, it is a weak but real signal that unvalidated tool creation can *reduce* task completion when the synthesis loop is not gated by a verification step. We treat this as a forward-looking signal rather than a verdict on the original CREATOR design; see Limitations on protocol adaptation.

This is the central empirical constraint in the current submission: the strict subset does not justify a claim that tool-creation protocols improve task completion. The benchmark contribution therefore rests on measurement, traceability, and artifact-level diagnostics rather than on a method win.

5.2 The audit view remains informative

The lack of a TC winner does not make the benchmark uninformative. It changes what the benchmark is showing. One-Shot and CREATOR-style create many tools (114 and 111 respectively), while No-Evolution and EvoSkill-style create none. ToolMaker-style creates fewer tools (18), consistent with a more conservative validation loop. These differences are invisible in a TC-only table.

Reuse precision also carries information that raw pass rates do not. The underlying counts are small but informative: One-Shot has 18 correct vs. 36 incorrect reuses out of 54 total (0.333), No-Evolution 9/18 out of 27 (0.333), EvoSkill-style 9/27 out of 36 (0.250), CREATOR-style 12/24 out of 36 (0.333), and ToolMaker-style 9/15 out of 24 (0.375). The denominators are tiny — this is why we present them as a directional signal, not a ranking — but they show the kind of decomposition the harness exposes: when a system uses prior artifacts, does that reuse coincide with success or with failure?

The positive contribution is therefore methodological. A TC-only benchmark would encourage a reader to rank five systems by a few percentage points of pass rate. EvolveTool-Bench instead asks what library state produced those pass rates. Did the system create tools? Did it reuse them? Did reuse help? Were unverified tasks excluded? Can the reader inspect the artifact? These questions are closer to the trustworthiness problem faced by long-running agents.

Worked trace: what the audit catches that TC misses. A concrete case illustrates the gap. In `data_transform_s1`, the system creates a single tool, `decode_abr_format`, to satisfy the gap task for the `abr_binary_decode` capability. The session's session-level outcome looks excellent: TC reaches 0.95, with the tool feeding into the variant, compose, and adversarial tasks that share that capability. Self-tests the agent wrote alongside the tool pass at 0.50. The audit layer, however, replays the same artifact against a held-out conformance suite tied to the capability (six unit inputs and six adversarial inputs that the agent never saw). Under that suite, the tool's correctness is 0.00: it passes every in-session task that reuses it, and fails every hidden test for the capability it claims to implement. The session-level reuse decomposition already hints at the problem – one of the three reuses is recorded as incorrect – but only the artifact-level re-evaluation makes the failure mode explicit. TC alone reports a near-perfect session; the audit reports a session that passed by structurally agreeing with its own tests. This is the kind of finding the framework is built to surface.[1]

5.3 Verifier coverage changes the interpretation

The strict pilot also illustrates why verifier coverage must be reported. If unverified tasks are credited by a length or plausibility heuristic, absolute TC can be inflated and protocol comparisons can move for the wrong reason. By restricting the main analysis to deterministic verifiers, we sacrifice benchmark coverage but gain measurement credibility.

The cost is clear: the API-orchestration domain is currently not part of the main strict TC analysis. That should be read as a limitation of the current release, not as a limitation of the benchmark idea. The framework already identifies what must be done next: complete API verifiers, align hidden tests by capability, and regenerate the same tables from the full manifest.

6 Discussion

Interpreting the pilot. The strict pilot should be read as an audit of an evaluation setting, not as a final ranking of agent designs. Its most important outcome is that the immediate TC comparison is not decisive. That makes the artifact layer more, not less, important: if several protocols land in the same TC range, researchers need to know whether those protocols created reusable code, accumulated unused artifacts, or relied on inline reasoning without leaving a library behind.

Implications for evaluation design. A benchmark for tool-evolving agents should make the evolving artifact inspectable. Generated tools are persistent objects: they can be reused, duplicated, retired, or accidentally made into dependencies for later failures. Recording creation, reuse, verifier status, and artifact summaries gives future work a sharper target than a single pass-rate column.

Design lessons for tool-evolving agents. The strict pilot suggests three concrete design lessons. First, tool creation alone is not evidence of progress; many tools can be created without improving verified task completion. Second, reuse needs a quality signal; raw reuse is ambiguous without correct/incorrect decomposition. Third, verifier coverage is a first-class benchmark property. A tool-evolving system is only as trustworthy as the tests and records used to decide what it has learned.

Using the benchmark constructively. The benchmark is most useful as a debugging instrument. A future method can use it to test a new promotion gate, deduplication policy, regression guard, or capability-specific verifier. The result to optimize is not only TC, but also whether the generated library becomes easier to reuse and safer to carry forward.

[1]The trace above is taken from a Code-Evol/Sonnet pilot in which per-tool source was preserved. The strict-pilot Haiku runs reported in Tables 3–4 record session-level and system-level summaries; aligning per-tool conformance to the same hidden-test manifest under the strict regime is the next artifact-hardening step.

7 Limitations

Verified subset. The strict statistical analysis covers 8 sessions and 51 verified task decisions per system pass, not the full 99-task benchmark. The API-orchestration session is excluded from main TC until deterministic verifiers are complete.

Pilot scale and single-model setting. The strict pilot uses one model (Claude Haiku 4.5) and three integer seeds. Several per-session outcomes are repeated across seeds, so standard errors primarily reflect session heterogeneity rather than broad model stochasticity. A smaller model is also where tool-creation protocols are most stress-tested: it is plausible that the absence of a TC improvement reflects synthesis quality at Haiku scale rather than a property of the protocols themselves. The strict pilot should therefore be read as evidence that the framework discriminates and that current TC differences are small, not as a verdict on tool-creation protocols in general. Larger multi-model runs are required before making stable protocol-ranking claims.

Protocol adaptations. EvoSkill-style, ToolMaker-style, and CREATOR-style are adaptations to a shared open-task harness, not native reproductions of the original systems. The original CREATOR and ToolMaker pipelines assume reference test suites or repository-grounded inputs that our open synthesis tasks do not supply; the `-style` suffix is deliberate, and the paper does not treat any underperformance in this pilot as a verdict on the original methods.

Hidden conformance tests. The benchmark design includes capability-aligned hidden tests for tool conformance. The main strict-subset results in this submission do not yet use a fully aligned conformance manifest, so we do not headline per-tool correctness numbers. Completing this alignment is the next artifact-hardening step.

Library-health composite. A composite score can be useful as a dashboard, but the current submission emphasizes decomposed metrics. Until the conformance and verifier maps are complete, a full ETS ranking would be premature.

8 Conclusion

Tool-evolving agents create a new evaluation target: the library they leave behind. A task-completion score remains necessary, but it is not enough to judge whether that library is reusable, stable, or auditable. EvolveTool-Bench provides a structured way to inspect this artifact through task roles, verifier-aware TC, reuse decomposition, and trace preservation.

The strict verified-subset pilot does not support a claim that any tested protocol improves task completion. It does, however, show why a TC-only view is too thin for this class of systems. The contribution is a diagnostic framework and a set of reusable practices for future research on tool-creating agents.

Reusable Evaluation Practices

We recommend that future benchmarks for tool-evolving agents:

(1) separate task success from generated-artifact behavior;
(2) report verifier coverage and exclude unverified tasks from TC claims;
(3) distinguish correct reuse from incorrect reuse;
(4) preserve generated artifacts, artifact summaries, and task/session traces;
(5) include regression and composition probes after library growth;
(6) evaluate library-management policies such as promotion, deduplication, and retirement.

References

[Alzubi et al.(2026)] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. 2026. EvoSkill: Automated Skill Discovery for Multi-Agent Systems. *arXiv preprint arXiv:2603.02766* (2026).

[Backlund and Petersson(2025)] Axel Backlund and Lukas Petersson. 2025. Vending-Bench: A Benchmark for Long-Term Coherence of Autonomous Agents. *arXiv preprint arXiv:2502.15840* (2025).

[Cai et al.(2023)] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2023. Large Language Models as Tool Makers. *arXiv preprint arXiv:2305.17126* (2023).

[Ellis et al.(2021)] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. 2021. DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation.*

[Li et al.(2026)] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, et al. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026).

[Liu et al.(2024)] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations.*

[Qian et al.(2023)] Cheng Qian, Chi Han, Yi R. Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023.*

[Qin et al.(2023)] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, et al. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv preprint arXiv:2307.16789* (2023).

[Singhal et al.(2024)] Manav Singhal, Tushar Aggarwal, Abhijeet Awasthi, Nagarajan Natarajan, and Aditya Kanade. 2024. NoFunEval: Funny How Code LMs Falter on Requirements Beyond Functional Correctness. *arXiv preprint arXiv:2401.15963* (2024).

[Wang et al.(2023)] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Advances in Neural Information Processing Systems* (2023).

[Wölflein et al.(2025)] Georg Wölflein, Dyke Ferber, Daniel Truhn, Ognjen Arandjelović, and Jakob Nikolas Kather. 2025. LLM Agents Making Agent Tools. *arXiv preprint arXiv:2502.11705* (2025).

[Xia et al.(2026)] Bowei Xia, Mengkang Hu, Shijian Wang, Jiarui Jin, Wenxiang Jiao, Yuan Lu, Kexin Li, and Ping Luo. 2026. Tool-Genesis: A Task-Driven Tool Creation Benchmark for Self-Evolving Language Agents. *arXiv preprint arXiv:2603.05578* (2026).

[Zhang et al.(2026)] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, et al. 2026. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. *arXiv preprint arXiv:2604.01687* (2026).